KainTayo

A Campus-Aware Visual Directory & Budget Meal Finder

Application Project Documentation

DCIT 26

GUTEAM

Contante, Jon Viktor B.

Emelo, Harry V.

Oracion, Samuel Antonio F.

January 2026

I. PROJECT OVERVIEW

KainTayo is a community-driven mobile application designed to assist Filipino students and working individuals in navigating financial constraints commonly experienced during “Petsa de Peligro”, the period before the next allowance or payday. The application addresses daily challenges such as budget limitations and decision fatigue by providing a crowd-sourced, location-based directory of affordable meals priced under ₱100. Users can explore nearby food options submitted and reviewed by fellow community members, ensuring both accessibility and reliability of information.

To further simplify meal choices, KainTayo features a gamified “Shake-to-Decide” function that randomly suggests a budget-friendly meal option based on the user’s location and preferences. This interactive feature reduces overthinking and helps users make quick, stress-free decisions. By combining practical financial assistance with community participation and playful engagement, KainTayo not only promotes smarter spending habits but also supports local food vendors, fostering a sustainable and inclusive food discovery ecosystem.

II. PROJECT SETUP

A. Prerequisites

To successfully run the KainTayo development environment, the following tools are required. Specific versions are noted to ensure compatibility.

- **Node.js (v18 LTS or higher):** The JavaScript runtime required to run the Expo bundler and the Express backend.
- **Git:** For version control and repository cloning.
- **Expo Go App:** Installed on a physical Android or iOS device for the primary testing workflow.
- **Visual Studio Code:** The recommended IDE with extensions for ES7+, Prettier, and TypeScript.
- **MongoDB Compass:** A GUI tool for visualizing and managing the database content locally.

B. Environment Setup

KainTayo utilizes a Monorepo structure. The setup involves installing the base tools, cloning the repository, and configuring the Client and Server independently.

Step 1: Tool Installation

1. Download and install **Node.js**. Verify installation: `node -v`
2. Download and install **Git**. Verify installation: `git --version`

Step 2: Cloning the Repository

Command	Description
`git clone <REPO_LINK>`	Clones the repository to your local machine.
`cd kaintayo-app`	Enter the project root directory.

Step 3: Backend Configuration (Server)

1. Navigate to the server: `cd server`
2. Install dependencies: `npm install`
3. Create a `.env` file in the `server/` root with these credentials:

Variable	Value
`PORT`	`5000`
`MONGO_URI`	`mongodb+srv://<user>:<password>@cluster.mongodb.net/kaintayo`
`JWT_SECRET`	`super_secret_key_123`

4. Start server: `npm run dev` (Runs on Port 5000)

Step 4: Frontend Configuration (Client)

1. Open a **new terminal**: `cd client`
2. Install dependencies: `npm install`
3. **Localhost Connection:** Update `client/constants/Config.ts` with your PC's Local IP:

Config	Code
`BASE_URL`	`const BASE_URL = 'http://192.168.1.X:5000/api/v1';`

4. Start app: `npx expo start`

Step 5: Running the App

- **Physical Device:** Scan the QR code with **Expo Go**.
- **Emulator:** Press `a` in the terminal to launch Android Emulator.

III. APP STRUCTURE

The project follows a Monorepo architecture combined with Vertical Slicing on the frontend. This structure ensures that features are decoupled, making the codebase scalable and easier to maintain.

A. Project Tree

```
kaintayo-app/
├── client/                 # React Native (Expo)
│   ├── app/                # Expo Router (File-based Navigation)
│   ├── components/         # Global/Reused UI (Button, Skelton)
│   ├── constants/          # Theme, Config
│   ├── features/           # VERTICAL SLICES (Domain Logic)
│   │   ├── admin/          # Admin Dashboard & Moderation
│   │   ├── auth/           # Login/Register & Context
│   │   ├── contribute/     # Add Meal/Place Forms
│   │   ├── directory/      # Home Feed, Filters, Search
│   │   ├── favorites/      # User Liked Places
│   │   ├── onboarding/     # Welcome Screens
│   │   ├── profile/        # User Dashboard & Settings
│   │   └── shake/          # Randomizer Game Logic
│   ├── hooks/              # Global Hooks (useDebounce)
│   └── lib/                # 3rd Party Config (Axios, SecureStore)
├── server/                 # Express API
│   ├── config/             # DB Connection Setup
│   ├── controllers/        # Request Handling
│   ├── middleware/             # Intermediate logic
│   ├── models/             # Mongoose Schemas (Place, Meal, User)
│   └── routes/             # API Endpoints
│   ├── schemas/            # Zod Validation Layers
│   ├── services/           # Business Logic
└── README.md
```

B. Directory Definitions

Frontend (Client)

Directory	Purpose
app/	Contains Expo Router files. Handles file-based navigation and screen layouts.
features/	Implements Vertical Slicing. Code is grouped by feature (e.g., `auth`, `feed`, `shake`) rather than file type, keeping related logic, UI, and state together.
components/	Contains global, reused UI elements reused across multiple features (e.g., CustomButtons, Loaders).
lib/	Contains third-party library configurations (e.g., Axios instance, TanStack Query client).
hooks/	Contains global custom hooks shared across the application.

Backend (Server)

Directory	Purpose
controllers/	Handles incoming HTTP requests and sends responses. Acts as the entry point for logic.
services/	Contains the core business logic. Separating this from controllers ensures code reusability and cleaner testing.
models/	Defines Mongoose schemas for data structure (User, Place, Meal).
schemas/	Contains Zod validation schemas to ensure API input integrity before processing.
middleware/	Handles intermediate logic like Authentication checks (JWT verification) and Error handling.

IV. CORE FEATURES

The KainTayo application is built around four primary pillars designed to solve the specific pain points of university students: budget constraints, decision fatigue, and data reliability.

1. Campus-Aware Visual Directory (Home Feed)

Expected Functionality: The Directory serves as the main entry point, providing a visually rich list of food establishments. Unlike generic maps, it filters content based on university-specific context (e.g., 'Inside Campus', 'Outside Campus'), strict budget limits, and categories.

User Interaction Flow:

1. **Budget Constraint:** User drags the Budget Slider to their current limit (e.g., ₱150). The list updates in real-time server-side, utilizing a 500ms Debounce to prevent API spam.
2. **Hybrid Sticky Header:** As the user scrolls, the Search Bar naturally scrolls away to maximize screen real estate, while the Filter Bar (Inside/Outside toggle + Chips) "sticks" to the top of the screen, ensuring filters are always accessible.
3. **Discovery:** User scrolls through "Hero Cards" displaying the establishment's vibe and price range. Content is loaded progressively via Infinite Scroll (10 items per page).

Key Components & Data Integration:

- **UI Components:** `DirectoryScreen` (Main View), `BudgetSlider` (Input), `PlaceCard` (Display), `StickyFilterHeader` (Navigation).
- **Data Hooks:** `useInfinitePlaces` (TanStack Query for pagination), `useDebounce` (Optimization).
- **API:** `GET /places` with query params `zoneMacro`, `maxPrice`, `search`.

2. Shake Randomizer (Decision Engine)

Expected Functionality: A gamified feature combating decision fatigue. Uses the device's Accelerometer to trigger a random selection from a filtered pool.

User Interaction Flow:

1. **Live Match Count:** The app queries the server in real-time to show how many places match the current criteria (e.g., "Shaking from 12 places").
2. **The Trigger:** User physically shakes the phone (detected via `expo-sensors`). Haptic feedback (Vibration) confirms the action.
3. **Suspense Phase:** A 5-second animation plays to build anticipation.
4. **Result Reveal:** The winner is displayed in a modal with options to "Let's Eat" (Navigation) or "Shake Again".

Key Components & Data Integration:

- **UI Components:** `ShakeScreen` (Sensor Consumer), `ShakeResultModal` (Winner Display), `ShakeAnimation`.
- **Data Hooks:** `useShake` (Manages Accelerometer subscription & logic).
- **API:** `GET /shake` (Server-side weighted randomization).

3. Smart Contribution (Wizard of Oz Strategy)

Expected Functionality: Allows community growth while maintaining quality. This feature employs a "Wizard of Oz" strategy: while the submission process feels seamless and automated to the contributor, the backend verification is manually handled by human administrators. New submissions remain hidden ("Pending") until explicitly approved, ensuring high data quality without the need for complex automated moderation algorithms.

User Interaction Flow:

1. **Duplicate Check:** User performs a search. If no match is found, they proceed to create a new place.
2. **Data Entry:** User interacts with a multi-step form:
    - **Place Details:** Name, Zone (Inside/Outside).
    - **Meal Details:** Title, Price, Half-Order availablity.
3. **Image Handling:** User selects a photo via `expo-image-picker`, which is uploaded directly to Cloudinary (Unsigned).

4. **Submission:** The form sends the data transactionally to the backend.

Key Components & Data Integration:

- **UI Components:** `ContributeScreen` (Form Layout), `AutocompleteInput` (Search), `ImageUploader` (Media).
- **Data Hooks:** `useMutation` (Optimistic Updates).
- **Service:** `contributeService.submitContribution` (Handles transactional `Create Place` -> `Add Meal` calls).

4. User System (Profile & Favorites)

Expected Functionality: Manages user identity and allows "Guest" users to browse while restricting "Write" actions to authenticated users.

User Interaction Flow:

1. **Guest Access:** Unauthenticated users can fully use the Directory and Shake features.
2. **Protected Actions:** If a guest attempts to 'Heart' a place or 'Submit' a meal, the action is intercepted to prompt login.
3. **Authentication:** User logs in/registers via JWT.
4. **Persistence:** The session is persisted securely and restored on app launch.

Key Components & Data Integration:

- **UI Components:** `AuthModal` (Login/Register Forms)
- **Context:** `AuthContext` (Global User State & Methods).
- **Storage:** `SecureStore` (Encrypted Token Persistence), `AsyncStorage` (Onboarding).

V. UI/UX DESIGN PRINCIPLES

The application interface is designed with clarity, consistency, and accessibility in mind to ensure a smooth and intuitive user experience. The following guidelines outline the UI/UX practices applied during development, directly reflecting the `theme.ts` and component architecture.

A. Color System & Accessibility

All colors are centralized in `client/lib/theme.ts` and verified to meet WCAG AA standards (minimum 4.5:1 contrast).

1. Brand Identity

Role	Token	Hex Code	Usage
Primary	`primary`	`#FF6B35`	Main Call-to-Action (Buttons), Active Tab Icons, Highlights
Accent	`secondary`	`#FDBA74`	Chips, Badges, Secondary Highlights

2. Functional (Semantic) Colors

Role	Token	Hex Code	Usage
Success	`success`	`#22c55e`	"Open Now" status, Success Toasts, Verified Prices
Error	`error`	`#ef4444`	Validation Errors, Destructive Actions (Delete)
Warning	`warning`	`#f59e0b`	"Pending Review" status, Network Issues
Info	`info`	`#3b82f6`	Informational Toasts

3. Surface & Text

Role	Token	Hex Code	Usage
Background	`background`	`#F5F5F5`	Global screen background (Neutral Light Gray)
Surface	`surface`	`#FFFFFF`	Cards, Modals, Bottom Sheets
Text Primary	`text.primary`	`#333333`	Headings, Body Text (High Contrast)
Text Secondary	`text.secondary`	`#666666`	Labels, Captions, Timestamps

B. Typography Scale

The app utilizes system-native fonts (San Francisco on iOS, Roboto on Android) to ensure zero-overhead loading and platform familiarity.

Role	Size (`theme.fontSizes`)	Weight	Usage
Hero	32px (`hero`)	Bold	Marketing Headers, Splash Screens
H1	24px (`header`)	Bold	Screen Titles (e.g., "Place Profile")
H2	20px (`xl`)	SemiBold	Card Titles, Modal Headers
Body	16px (`md`)	Regular	Standard content, Descriptions
Label	14px (`sm`)	Medium	Button Text, Input Labels
Caption	12px (`xs`)	Regular	Timestamps, Helper Text

C. Layout & Spacing

1. 8pt Grid System

All spacing and sizing follows a strict 8-point grid to ensure rhythm and consistency.

- `xs` **(8px):** Internal component padding (e.g., inside a Chip).
- `sm` **(12px):** Compact spacing (tweaked for dense lists).
- `md` **(16px):** Standard screen padding and card gaps.
- `lg` **(24px):** Section separators.
- `xl` **(32px):** Major layout breaks.

2. Component Radius

- **Cards:** 12px (`radius.lg`) - Creates a modern, friendly aesthetic.
- **Buttons:** 8px (`radius.md`) - Standard interactive elements.
- **Inputs:** 8px (`radius.md`) - Consistent with buttons.

3. Touch Targets

- **Minimum Size:** All interactive elements (Buttons, Icons, List Items) have a minimum touch target of 44x44dp to prevent "fat finger" errors.

D. Navigation Structure

The app uses Expo Router (File-based routing) to manage navigation stacks.

1. **Tab Navigator (Main Layout):**
    - **Food:** Home feed & filtering.
    - **Shake:** Randomizer feature.
    - **Contribute:** Submission forms.
    - **Profile:** User settings & dashboard.
2. **Stack Navigator (Overlays):**
    - **Place Details:** Pushed onto the stack when a card is tapped.
    - **Auth Modal:** Presented as a modal over the current context.
    - **Favorites Screen:** Displays the user’s favorite places.
    - **My Contributions Screen:** Presents the pending contributions of a user.
    - **Admin Panel:** Shows as a Profile menu for “admin” users.
    - **Onboarding Screen:** Displays when a user enters the app for the first time.

E. User Feedback Elements

1. **Alerts**

   Native Alerts are used sparingly for critical, irreversible actions that require explicit user confirmation, such as deleting a contribution or removing a favorite.

2. **Custom Toasts**

   Implemented via `ToastContext`, replacing intrusive alerts with non-blocking feedback for success and error messages.

3. **Loading Skeletons**

   Implemented via `PlaceCardSkeleton` to prevent layout shifts.

   - **Buffered Loading:** Skeletons persist for a minimum of 500ms to avoid a "flash" effect if the data loads too quickly.

4. **Empty States**

   Custom illustrations and helpful text (e.g., "No places found within ₱50") guide users when filters yield no results, preventing dead ends.

VI. EFFICIENT FUNCTIONALITY

The KainTayo development team prioritizes efficiency by optimizing data computation, reusing core UI components, and minimizing unnecessary network requests. This section outlines the architectural decisions and best practices implemented to achieve a high-performance, scalable application.

A. Code Structure & Reusability

1. **Vertical Slicing Architecture**

   Instead of grouping files by type (e.g., all controllers in one folder), the frontend uses Vertical Slicing. Features are self-contained modules, ensuring that related logic, UI, and state are co-located. This reduces cognitive load and makes the codebase easier to maintain.

2. **Feature-based and Reusable Components**

   UI elements are organized into feature-based components and shared UI utilities.

   - **Shared UI:** `BudgetSlider`, `FilterModal`, `FormError` (Reused components).
   - **Feature Components:** `PlaceCard`, `ShakeButton`, `LoginForm`, (Dedicated UI).
   - **Screens:** `DirectoryScreen`, `ShakeScreen` (Page layouts).

   By centralizing common UI elements, we ensure consistent styling and reduce code duplication.

B. Data Handling & Optimization

1. **Server-Side Price Range Caching**

   Calculating the "Price Range" (Min - Max) for a place with 50+ meals client-side would be expensive.

   - **Optimized approach:** When a meal is added (e.g., ₱50), the backend automatically updates the parent Place's `minPrice` and `maxPrice` fields.
   - **Benefit:** The Budget Slider filters the Directory instantly via a simple MongoDB query (`minPrice <= UserBudget`) without aggregate computation at runtime.

2. **Debounced API Requests**

   To prevent server overload and "flickering" UI, the Budget Slider implements a 500ms Debounce (`useDebounce` hook).

   - **Mechanism:** API requests are only sent after the user *stops* sliding for 500ms.
   - **Benefit:** Reduces server load by ~80% during active filtering.

3. **Infinite Scrolling (Pagination)**

   The Directory loads data in small chunks (10 items per page) using TanStack Query's `useInfiniteQuery`.

   - **Benefit:** Keeps the app fast and responsive even with thousands of listings, as only the visible data is rendered.

C. State Management

1. **Server State (TanStack Query)**

   We distinguish between "Client State" and "Server State".

   - **Server State:** Directory lists, Place details, and Profile stats are managed by TanStack Query. It handles caching, background refetching, and deduping requests automatically.

2. **Global Client State (Context API)**

   For strictly client-side global data, we use React Context.

   - **`AuthContext`:** Manages User Identity (JWT), Login/Logout methods, and Loading State.
   - **`ToastContext`:** Manages global feedback notifications.

D. Robust Validation & Error Handling

1. **Zod Validation**

   We strictly validate all inputs using Zod schemas on the backend (`server/schemas/placeSchema.ts`).

   - **Security:** Prevents "overposting" attacks and ensures data integrity (e.g., verifying price is a positive number).
   - **Feedback:** Returns clear error messages to the client which are mapped to UI Toasts.

2. **User Feedback Loop**

   - **Skeletons:** `PlaceCardSkeleton` provides a buffered loading state (minimum 500ms) to prevent layout shifts.
   - **Toasts:** Non-blocking notifications confirm actions ("Saved to Favorites") or report errors ("Network Error").

E. Image Optimization

1. **Cloudinary Integration**

   We use Cloudinary for image hosting and delivery.

   - **Unsigned Uploads:** Clients upload images directly to Cloudinary, bypassing the main server's bandwidth limits.
   - **On-the-fly Transformation:** Images are auto-compressed and resized (e.g., `w_400,q_auto`) via URL parameters to ensure fast loading on mobile networks.

VII. FUTURE IMPROVEMENTS

As the KainTayo user base grows, the application can expand to offer more granular data, social features, and accessibility options. The following features are recommended for the next development phase, based on user feedback and scalability requirements.

A. Map Integration

Currently, the app uses a list-based Directory. Integrating a map view will help users visualize walking distances, which is critical for decision-making on a large campus.

- **Proposed Feature:** A toggle button on the Home Directory will allow users to switch between "List View" and "Map View". In Map View, establishments will be plotted as pins, color-coded by price range (e.g., Green for <₱50), enabling users to spot the nearest affordable meal instantly.

B. Social Reviews & Ratings

To increase trust and community engagement, the system should move beyond simple "Favorites" to detailed reviews.

- **Proposed Feature:** Authenticated users will be able to leave a 1–5 Star Rating and written comments on specific meals. The Place Profile will display the aggregate "Average Rating," giving new users confidence in the quality and portion sizes of the establishment before visiting.

C. Real-Time Status Indicators

Students often walk to a canteen only to find it closed. Real-time status updates would eliminate this frustration.

- **Proposed Feature:** Place Cards will display "Open Now" (Green) or "Closed" (Red) badges based on the current time and the establishment's operating hours. If a vendor closes unexpectedly, a "Temporary Closed" status can be manually triggered.

D. Community Reporting System

To maintain data accuracy as the directory grows, the community needs a way to flag outdated or incorrect information.

- **Proposed Feature:** A "Report Issue" button will be available on every Place Profile. Users can select from categories such as "Price Changed," "Place Closed," or "Inappropriate Content." These reports will be sent to the Admin Dashboard for moderation, ensuring the "Wizard of Oz" data remains reliable without requiring constant manual surveillance by the developers.

VIII. REFERENCES

The development of KainTayo utilized the following documentation, frameworks, and design standards to ensure industry-grade performance and usability.

A. Technologies & Frameworks

1. React Native Documentation. (n.d.). Core Components and APIs. Retrieved from https://reactnative.dev/docs/components-and-apis
2. Expo Documentation. (n.d.). Expo Router and File-Based Routing. Retrieved from https://docs.expo.dev/router/introduction/
3. MongoDB Atlas. (n.d.). MongoDB Documentation & Node.js Drivers. Retrieved from https://www.mongodb.com/docs/drivers/node/current/
4. Express.js. (n.d.). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from https://expressjs.com/

B. Libraries & Tools

5. TanStack Query (React Query). (n.d.). State Management and Data Fetching. Retrieved from https://tanstack.com/query/latest
6. Zod. (n.d.). TypeScript-first schema validation with static type inference. Retrieved from https://zod.dev/
7. Cloudinary. (n.d.). Image Transformation and Optimization API. Retrieved from https://cloudinary.com/documentation

8. Expo Sensors. (n.d.). Accelerometer and Motion APIs. Retrieved from https://docs.expo.dev/versions/latest/sdk/accelerometer/

C. Design & Conceptual References

9. W3C Web Accessibility Initiative (WAI). (n.d.). Web Content Accessibility Guidelines (WCAG) 2.1. Retrieved from https://www.w3.org/TR/WCAG21/
10. Hick's Law. (n.d.). Laws of UX. Retrieved from https://lawsofux.com/hicks-law/
11. Nielsen Norman Group. (n.d.). Mobile UX: Navigation Patterns. Retrieved from https://www.nngroup.com/articles/mobile-navigation-patterns/